

function

```
num = int(input("Enter a number: "))  
count = 0
```

```
while num > 0: true  
    count = count + 1  
    num = num // 10
```

```
print("No of digits:", count)
```

num = 12345 ← (5)

count = 0

Count = 1 ✓

num = 12345 // 10 → 1234

1234 ←

10) 12345

10

23

20

34

30

45

40

5

Count = 3

num = 123 // 10
= 12

Count = 2

num =
1234 // 10
= 123

Count = 4

num = 12 // 10
= 1

Count = 5

num = 1 // 10

function → reusable code in Python
that performs a
specific task

① User defined function

↳ functions are defined by the user to perform a specific task.

② Built in function

`len ( )`
`max ( )`
`min ( )`
`sum ( )`

↳ functions that are pre-defined in Python

```
# function definition
def add(a, b):
    return a + b *
```

```
# function calling
total = add(10, 20)
print(total)
```

↘
30

```
# function definition
def add(a, b):
    print(a + b) *
```

```
# function calling
add(10, 20)
```

↘
30

→ parameters

arguments vs parameters

how do we define any user-defined function

*args vs **kwargs

return vs print
 to store the value of the function it is used to display the output in the console

① Palindrome → function which is used to check whether the given string is palindromic or not

mayan ✓
 mom

② `def is_armstrong(n):`
Write your logic

`n` → 153 → num_digits = 3

$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$

$$\begin{array}{r}
 123 \\
 \hline
 1^3 + 2^3 + 3^3 \\
 1 + 8 + 27 \\
 = 36
 \end{array}$$

equal to the sum of digits

Not a Armstrong number

yes

$$\begin{array}{r}
 370 \longrightarrow 3^3 + 7^3 + 0^3 \\
 = 27 + 343 + 0 \\
 = 370
 \end{array}$$

xx

Recursion $\longrightarrow$ Recursive function

factorial of a given number

A recursive function is a function that solves a problem by calling itself, breaking down complex tasks into smaller, identical subproblems until a simple "base case" is reached, which stops the recursion, preventing infinite loops

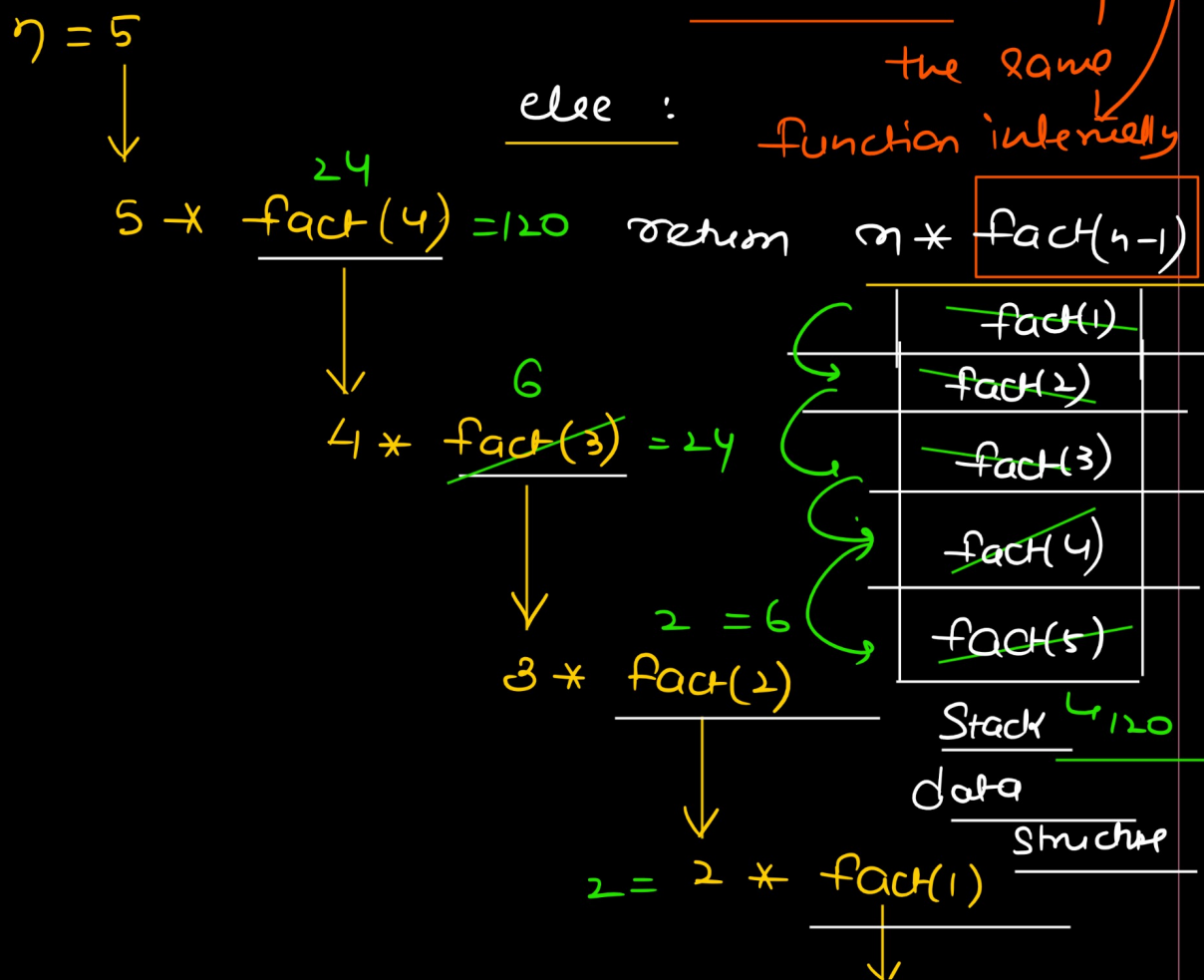
$$\text{factorial} \rightarrow 5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$5! = 120$$

base case condition $\leftarrow \begin{cases} 0! \rightarrow 1 \\ 1! \rightarrow 1 \end{cases}$

def fact(n):
 if n == 0 or n == 1:
 return 1
 else:
 return n * fact(n-1)

calling the same function internally



Scope

1

Scope refers to the region of a program where a variable is visible and can be accessed. Python has two main scopes: global scope and local scope

Global Scope: Variables defined outside any function are considered to be in the global scope. These variables can be accessed from anywhere in the program, including within functions.

def outer_function(x, y):

def add(a, b):
return a + b

✓
3 4

→ add

def multiply(a, b):
return a * b

✓

→ multiplication

def calculate(operation_type, a, b):

if operation_type == "add":

return add(a, b) ✓

elif operation_type == "multiply":

return multiply(a, b) ✓

else:

return "invalid operation"

return calculate("add", x, y), calculate("multiply", x, y)

} → calculate

add ✓

multiply ✓

invalid operation ✓

result = outer_function(3, 4)
print(result)

function call

function call

→ (add, 3, 4), (multiply, 3, 4)

(7) 12

Map $\rightarrow$ mapping of original
value to a
new set of values

Reduce $\rightarrow$ combining operation